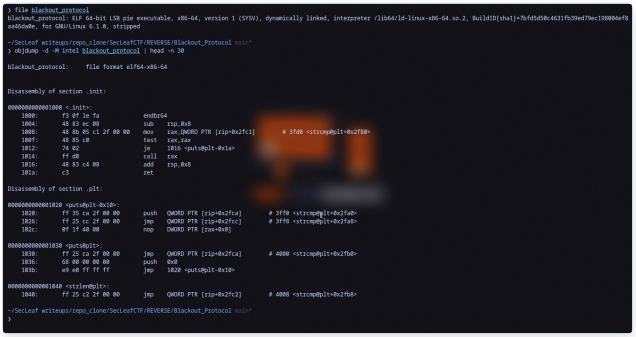
Blackout Protocol

Reverse Engineering



I started by checking the challenge archive containing stripped ELF64 executable `blackout_protocol`, a 16 KB raw memory dump `retained.mem`, and network capture `transmission.pcap`.

Loading the binary into GDB and objdump, I analyzed the validation loop: it parsed 192 candidate packets, authenticated 32 valid transmissions via CRC32, and initialized a 64-bit seed.

The binary executed a 52-round Substitution-Permutation Network (SPN) state engine against a 48-byte candidate input. Because every layer (S-Box, linear permutation, and key mixing) was invertible, I stepped backwards through all 52 rounds to recover the input.

Finally, I tested the recovered flag against the executable / service:

`python3 Blackout_Protocol.py`

Authentication passed: 32 packet trace validated.

Flag Captured

`SecLeaf{n01s3_1n_th3_w1r35_4nd_4ft3r1m4g3s!!!!!}`

BY:JOKER_53